

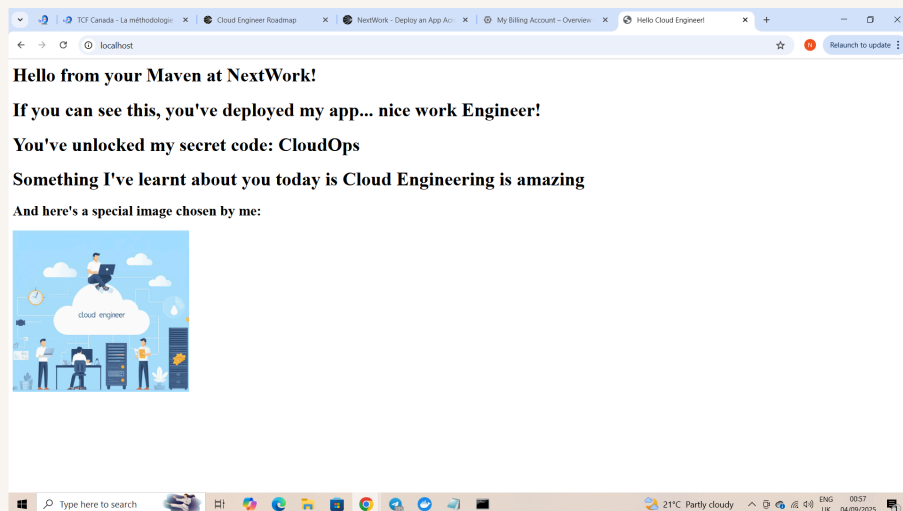


nextwork.org

Deploy an App Across Accounts



Nchindo Boris





Introducing Today's Project!

In this special multiplayer project, I'm working with a buddy to; - Build a Docker container image for my custom app. - Store my image securely in Amazon ECR (Elastic Container Registry). - Share container images with my project buddy and launch each other's apps!

What is Amazon ECR?

Amazon Elastic Container Registry (ECR) is a fully managed container registry service provided by AWS that allows you to securely store, manage, and deploy Docker and OCI container images. In this project, I used Amazon ECR (Elastic Container Registry) to push my Docker image. storing it securely in a private repository.

One thing I didn't expect...

I didn't expect setting up and attaching several policies and permissions to successfully deploy my app.

This project took me...

This project took me 2 hours and 30 minutes to complete.



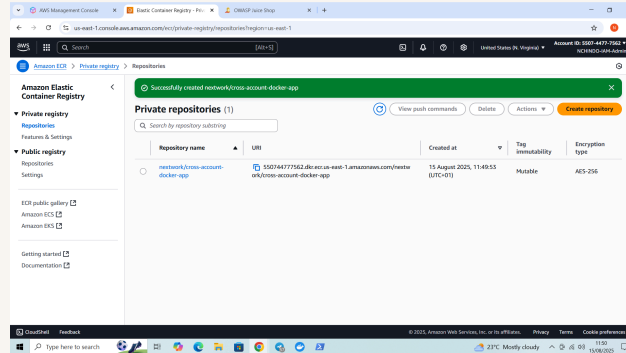
Creating a Docker Image

I set up a Dockerfile and an index.html file. Both are necessary because the Dockerfile defines the container setup using the Nginx image, while the index.html file contains the custom web content to be served. Together, these files create a functional Docker image that serves my custom web page when deployed.

My Dockerfile instructs Docker to use the latest official Nginx image as the base and copies my custom index.html file into the default Nginx web directory (/usr/share/nginx/html/). This setup ensures that when the container runs, it serves my personalized web page instead of the default Nginx page

I also set up an ECR repository

ECR stands for Elastic Container Registry. It is important because it is a fully managed container registry that makes it easy for you to store, manage, and deploy your container images.





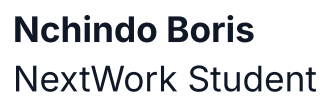
Set Up AWS CLI Access

AWS CLI can let me run ECR commands

AWS CLI is a powerful tool that lets you manage your AWS services from your terminal. Instead of having to use the AWS Management Console, you can now run text commands from your local machine. The CLI asked for my credentials because AWS requires every request to be authenticated to confirm your identity and determine what actions you are allowed to perform. When you run `aws configure`, you provide your access key, secret key, and region, which the CLI uses to sign and authorize your requests to AWS. Without these credentials, you can't use the CLI to push images or perform other operations, because AWS needs to know who is making the request and what permissions they have.

To enable CLI access, I set up a new IAM user with the permission policy : Amazon EC2Container RegistryFullAccess. I also created an access key for this user, which means I can now make programmatic requests to AWS services like pushing images to Amazon ECR using the AWS CLI. The access key consists of an access key ID and a secret access key, both of which are required for authentication. Without these credentials, the CLI cannot authorize my requests to AWS services.

I configured the AWS CLI by running the `aws configure` command in my terminal. This command prompted me to enter my AWS Access Key ID, Secret Access Key, default region, for the output format I left a blank and press Enter on your keyboard. The default format for the terminal responses from your AWS commands is JSON.



The screenshot shows a Windows terminal window titled "Command Prompt - msfconsole". The terminal output is as follows:

```
Microsoft Windows [UTF-8: 1252]
(c) Microsoft Corporation. All rights reserved.

C:\Users\B0752>msfconsole --version
msfconsole 4.12.0 Python 2.7.14 Windows/64 msfconsole

C:\Users\B0752>docker run --rm --name msfconsole -p 4444 -e perl -i msfconsole
Password was requested at this time.

C:\Users\B0752>msfconsole
msfconsole 4.12.0
```

The terminal window has a standard Windows taskbar at the bottom with the search bar, task view button, and several pinned application icons. The system tray shows the date and time as 11:27 on 15/06/2025.



Pushing My Image to ECR

Push commands are the set of Docker and AWS CLI commands used to upload (or "push") your local Docker images to an Amazon Elastic Container Registry (ECR) repository. This process makes your container images available in the cloud for deployment and sharing.

There are three main push commands

To authenticate Docker with my ECR repo, I used the command `aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 550744777562.dkr.ecr.us-east-1.amazonaws.com`. This command is used to retrieve an authentication token and authenticate your Docker client to your registry.

To push my container image, I ran the command `docker tag nextwork/cross-account-docker-app:latest 550744777562.dkr.ecr.us-east-1.amazonaws.com/nextwork/cross-account-docker-app:latest`. Pushing means uploading the local Docker image to a container registry making the image available for others to pull and deploy.

When I built my image, I tagged it with the label `latest`. This means, if you make updates to your code, you can rebuild the image, tag it as `latest`, and any deployments that reference `latest` will always pull the most up-to-date version.



Resolving Permission Issues

When I tried pulling my project buddy's container image for the first time, I saw the error Error response from daemon: failed to resolve reference "721605076466.dkr.ecr.af-south-1.amazonaws.com/maven/cross-account-docker-app:latest": pull access denied, repository does not exist or may require authorization: authorization failed: no basic auth credentials This was because Docker couldn't pull a Docker image from Amazon ECR.

To resolve each other's permission errors, my buddy and I updated our respective Amazon ECR repository policies to grant each other's IAM Users the necessary permissions to pull images. This involved adding each other's IAM User ARN under the "Principal" section of the repository policy and allowing actions such as `ecr:GetDownloadUrlForLayer`, `ecr:BatchGetImage`, and `ecr:BatchCheckLayerAvailability`. By saving these changes, we enabled secure cross-account access, allowing us to pull and use each other's container images as needed



```
C:\Windows\System32\cmd.exe
Name      Value      Type      Location
----      -
profile    <not set>  None      None
access_key *****B0S shared-credentials-file
secret_key *****U0I shared-credentials-file
region     us-east-1  config-file ~/.aws/config

C:\Users\BORIS\Desktop\DockeRCR>aws ecr get-login-password --region af-south-1 | docker login --username AWS --password-stdin 721605076466.dkr.ecr.us-east-1.amazonaws.com
Password was unexpected at this time.

C:\Users\BORIS\Desktop\DockeRCR>aws ecr get-login-password --region af-south-1 | docker login --username AWS --password-stdin 721605076466.dkr.ecr.af-south-1.amazonaws.com
Password was unexpected at this time.

C:\Users\BORIS\Desktop\DockeRCR>aws ecr get-login-password --region af-south-1 | docker login --username AWS --password-stdin 721605076466.dkr.ecr.af-south-1.amazonaws.com
An error occurred (UnrecognizedClientException) when calling the GetAuthorizationToken operation: The security token included in the request is invalid.
Error: Cannot perform an interactive login from a non TTY device

C:\Users\BORIS\Desktop\DockeRCR>aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 721605076466.dkr.ecr.us-east-1.amazonaws.com
Password was unexpected at this time.

C:\Users\BORIS\Desktop\DockeRCR>aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 721605076466.dkr.ecr.us-east-1.amazonaws.com
Login Succeeded

C:\Users\BORIS\Desktop\DockeRCR>docker pull 721605076466.dkr.ecr.af-south-1.amazonaws.com/maven/cross-account-docker-app
Using default tag: latest
Error response from daemon: failed to resolve reference "721605076466.dkr.ecr.af-south-1.amazonaws.com/maven/cross-account-docker-app:latest": pull access denied, repository does not exist or may require authorization: authorization failed: no basic auth credentials

C:\Users\BORIS\Desktop\DockeRCR>
```



Deploying the App

I used Elastic Beanstalk to deploy my application. When setting up the Elastic Beanstalk environment, I set up configurations to determine how the application runs and how AWS resources are managed during deployment.

While setting up for deployment, I created a new IAM role for Elastic Beanstalk called `ecrinstanceRole`. This role has permission to pull container images from Amazon ECR by attaching the Amazon EC2Container Registry ReadOnly policy. I also gave Elastic Beanstalk access to ECR by attaching the same policy to its service role, ensuring it can pull the image during deployment

The `Dockerrun.aws.json` file is a configuration file used by AWS Elastic Beanstalk to define how your Docker container should be deployed and run. My file tells Elastic Beanstalk which Docker image to use, the ports to expose, and any additional settings such as environment variables, commands, or volumes needed for the application to function properly.



```
DockerRun.aws - Notepad
File Edit Format View Help
{
  "AwsEBSDockerrunVersion": "1",
  "Image": {
    "Name": "721695876466.dkr.ecr.us-east-1.amazonaws.com/maven/cross-account-docker-app",
    "Update": "true"
  },
  "Ports": [
    {
      "containerPort": "80"
    }
  ]
}
```

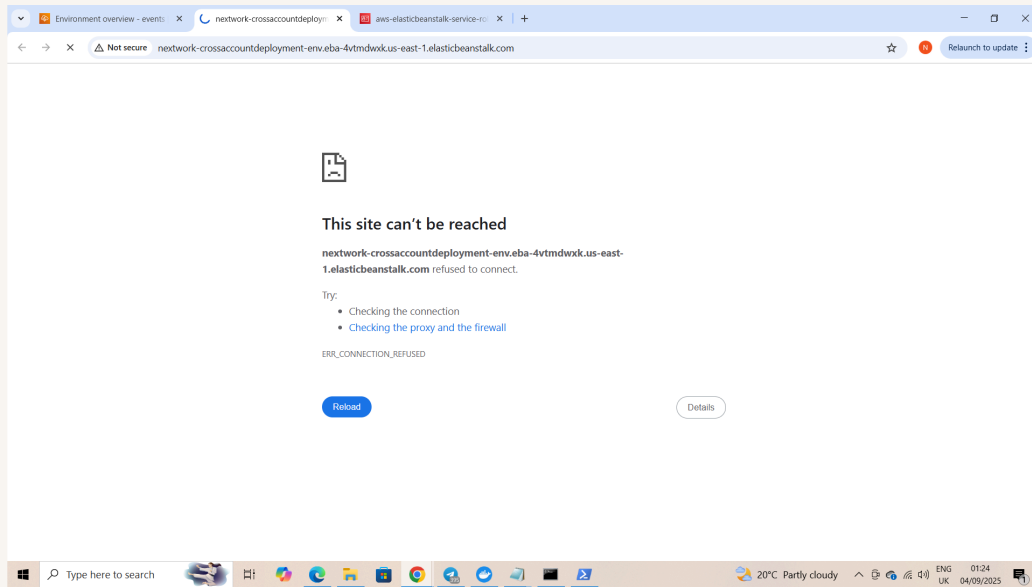
Ln 14, Col 1 100% Windows (CRLF) UTF-8
21°C Partly cloudy ENG 01:06
UK 04/09/2025



Resolving Deployment Issues

When I visited my environment's domain, I initially saw the error "site can't be reached". This was because the necessary permissions to pull the Docker image from Player A's ECR repository were missing.

To fix the permissions error, my buddy and I updated our ECR repository policies by adding each other's `ecrinstanceRole` and `aws-elasticbeanstalk-service-role` ARNS to the policy JSON. Previously, my ECR repository's permission settings did not allow Elastic Beanstalk or EC2 instances in either account to pull images during environment setup and scaling, which resulted in access errors.





nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

